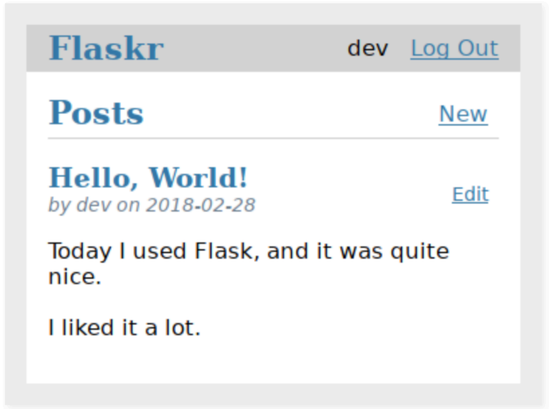
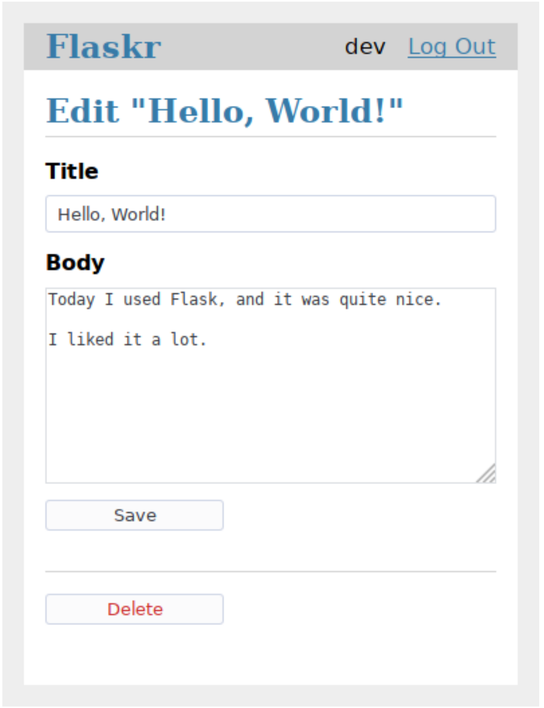
Flask

https://flask.palletsprojects.com/en/2.2.x/tutorial/





Directory structure

```
/home/user/Projects/flask-tutorial
├── flaskr/
│   ├── __init__.py
│   ├── db.py
│   ├── schema.sql
│   ├── auth.py
│   ├── blog.py
│   ├── templates/
│   │   ├── base.html
│   │   ├── auth/
│   │   │   ├── login.html
│   │   │   └── register.html
│   │   └── blog/
│   │       ├── create.html
│   │       ├── index.html
│   │       └── update.html
│   └── static/
│       └── style.css
├── tests/
│   ├── conftest.py
│   ├── data.sql
│   ├── test_factory.py
│   ├── test_db.py
│   ├── test_auth.py
│   └── test_blog.py
├── venv/
├── setup.py
└── MANIFEST.in
```

flaskr/__init__.py¶

```
flask --app flaskr --debug run
```

```
* Serving Flask app "flaskr"
* Debug mode: on
* Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)
* Restarting with stat
* Debugger is active!
* Debugger PIN: nnn-nnn-nnn
```

```python
import os

from flask import Flask


def create_app(test_config=None):
    # create and configure the app
    app = Flask(__name__, instance_relative_config=True)
    app.config.from_mapping(
        SECRET_KEY='dev',
        DATABASE=os.path.join(app.instance_path, 'flaskr.sqlite'),
    )

    if test_config is None:
        # load the instance config, if it exists, when not testing
        app.config.from_pyfile('config.py', silent=True)
    else:
        # load the test config if passed in
        app.config.from_mapping(test_config)

    # ensure the instance folder exists
    try:
        os.makedirs(app.instance_path)
    except OSError:
        pass

    # a simple page that says hello
    @app.route('/hello')
    def hello():
        return 'Hello, World!'

    return app
```

flaskr/db.py

```
DROP TABLE IF EXISTS user;
DROP TABLE IF EXISTS post;

CREATE TABLE user (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  username TEXT UNIQUE NOT NULL,
  password TEXT NOT NULL
);

CREATE TABLE post (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  author_id INTEGER NOT NULL,
  created TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  title TEXT NOT NULL,
  body TEXT NOT NULL,
  FOREIGN KEY (author_id) REFERENCES user (id)
);
```

```
import sqlite3

import click
from flask import current_app, g


def get_db():
    if 'db' not in g:
        g.db = sqlite3.connect(
            current_app.config['DATABASE'],
            detect_types=sqlite3.PARSE_DECLTYPES
        )
        g.db.row_factory = sqlite3.Row

    return g.db


def close_db(e=None):
    db = g.pop('db', None)

    if db is not None:
        db.close()
```

flaskr/db.py

```
DROP TABLE IF EXISTS user;
DROP TABLE IF EXISTS post;

CREATE TABLE user (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  username TEXT UNIQUE NOT NULL,
  password TEXT NOT NULL
);

CREATE TABLE post (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  author_id INTEGER NOT NULL,
  created TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  title TEXT NOT NULL,
  body TEXT NOT NULL,
  FOREIGN KEY (author_id) REFERENCES user (id)
);
```

```
def create_app():
    app = ...
    # existing code omitted

    from . import db
    db.init_app(app)

    return app
```

```
$ flask --app flaskr init-db
Initialized the database.
```

```
import sqlite3

import click
from flask import current_app, g


def get_db():
    if 'db' not in g:
        g.db = sqlite3.connect(
            current_app.config['DATABASE'],
            detect_types=sqlite3.PARSE_DECLTYPES
        )
        g.db.row_factory = sqlite3.Row

    return g.db


def close_db(e=None):
    db = g.pop('db', None)

    if db is not None:
        db.close()

def init_app(app):
    app.teardown_appcontext(close_db)
    app.cli.add_command(init_db_command)
```

flaskr/auth.py¶

```
import functools

from flask import (
    Blueprint, flash, g, redirect, render_template, request, session, url_for
)
from werkzeug.security import check_password_hash, generate_password_hash

from flaskr.db import get_db

bp = Blueprint('auth', __name__, url_prefix='/auth')
```

```
def create_app():
    app = ...
    # existing code omitted

    from . import auth
    app.register_blueprint(auth.bp)

    return app
```

flaskr/auth.py¶

```
import functools

from flask import (
    Blueprint, flash, g, redirect,
render_template, request, session, url_for
)
from werkzeug.security import
check_password_hash, generate_password_hash

from flaskr.db import get_db

bp = Blueprint('auth', __name__,
url_prefix='/auth')
```

```
@bp.route('/register', methods=('GET', 'POST'))
def register():
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']
        db = get_db()
        error = None

        if not username:
            error = 'Username is required.'
        elif not password:
            error = 'Password is required.'

        if error is None:
            try:
                db.execute(
                    "INSERT INTO user (username, password) VALUES (?, ?)",
                    (username, generate_password_hash(password)),
                )
                db.commit()
            except db.IntegrityError:
                error = f"User {username} is already registered."
            else:
                return redirect(url_for("auth.login"))

        flash(error)

    return render_template('auth/register.html')
```

flaskr/auth.py¶

```
@bp.before_app_request
def load_logged_in_user():
    user_id = session.get('user_id')

    if user_id is None:
        g.user = None
    else:
        g.user = get_db().execute(
            'SELECT * FROM user WHERE id
= ?', (user_id,)
        ).fetchone()
```

```
@bp.route('/logout')
def logout():
    session.clear()
    return redirect(url_for('index'))
```

```
@bp.route('/login', methods=('GET', 'POST'))
def login():
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']
        db = get_db()
        error = None
        user = db.execute(
            'SELECT * FROM user WHERE username = ?', (username,)
        ).fetchone()

        if user is None:
            error = 'Incorrect username.'
        elif not check_password_hash(user['password'], password):
            error = 'Incorrect password.'

        if error is None:
            session.clear()
            session['user_id'] = user['id']
            return redirect(url_for('index'))

        flash(error)

    return render_template('auth/login.html')
```

flaskr/auth.py¶

```
def login_required(view):
    @functools.wraps(view)
    def wrapped_view(**kwargs):
        if g.user is None:
            return
redirect(url_for('auth.login'))

        return view(**kwargs)

    return wrapped_view
```

```
@bp.route('/logout')
def logout():
    session.clear()
    return redirect(url_for('index'))
```

```
@bp.route('/login', methods=('GET', 'POST'))
def login():
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']
        db = get_db()
        error = None
        user = db.execute(
            'SELECT * FROM user WHERE username = ?', (username,)
        ).fetchone()

        if user is None:
            error = 'Incorrect username.'
        elif not check_password_hash(user['password'], password):
            error = 'Incorrect password.'

        if error is None:
            session.clear()
            session['user_id'] = user['id']
            return redirect(url_for('index'))

        flash(error)

    return render_template('auth/login.html')
```

flaskr/templates/base.html¶

```
<!doctype html>
<title>{% block title %}{% endblock %} - Flaskr</title>
<link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
<nav>
  <h1>Flaskr</h1>
  <ul>
    {% if g.user %}
      <li><span>{{ g.user['username'] }}</span>
      <li><a href="{{ url_for('auth.logout') }}">Log Out</a>
    {% else %}
      <li><a href="{{ url_for('auth.register') }}">Register</a>
      <li><a href="{{ url_for('auth.login') }}">Log In</a>
    {% endif %}
  </ul>
</nav>
<section class="content">
  <header>
    {% block header %}{% endblock %}
  </header>
  {% for message in get_flashed_messages() %}
    <div class="flash">{{ message }}</div>
  {% endfor %}
  {% block content %}{% endblock %}
</section>
```

flaskr/templates/auth/register.html¶

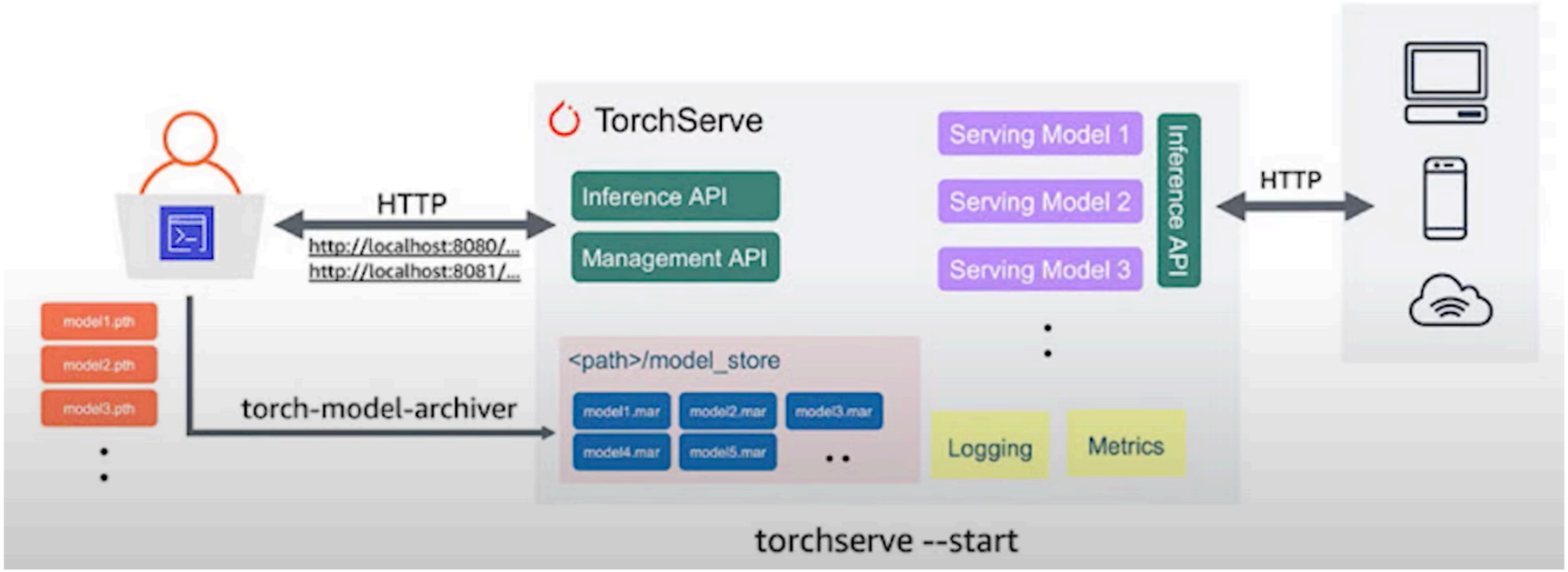
```
{% extends 'base.html' %}

{% block header %}
  <h1>{% block title %}Register{% endblock %}</h1>
{% endblock %}

{% block content %}
  <form method="post">
    <label for="username">Username</label>
    <input name="username" id="username" required>
    <label for="password">Password</label>
    <input type="password" name="password" id="password" required>
    <input type="submit" value="Register">
  </form>
{% endblock %}
```

Model serving

- Kaj je model serving?
- Kaj je torch serve?
- Mini demo

Kaj je model serving

- **Model serving** je proces s katerim še "natreniran" model ga naredimo dostopen za "širšo" uporabo



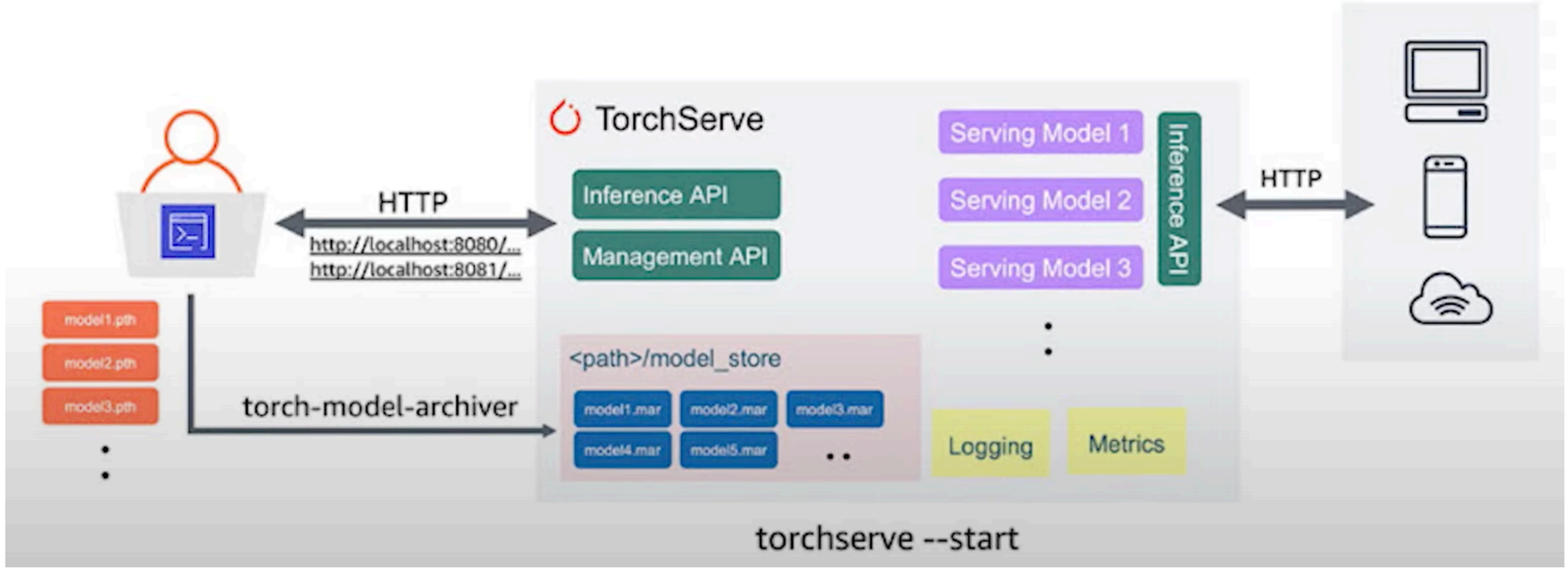
Kaj je model serving

- **TorchServe** ima "boilerplate" strukturo, ki omogoča osnovne tipe ML nalog:
  - Klasifikacija slik
  - Zaznavanje objektov
  - Text itd...

```python
import torch
from ts.torch_handler.base_handler import BaseHandler

class MyModelHandler(BaseHandler):
    def initialize(self, context):
        # get GPU status & device handle
        # load model & supporting files (vocabularies etc.)

    def preprocess(self, data):
        # put incoming data into tensor
        # transform as needed for your model

    def inference(self, context):
        # do predictions

    def postprocess(self, output):
        # process inference output, e.g. extracting top K
        # package output for web delivery
```

TorchServe

- Arhiv modelov
- InferenceAPI loading
- Konfiguracija in kontrola Managment API
- Logs

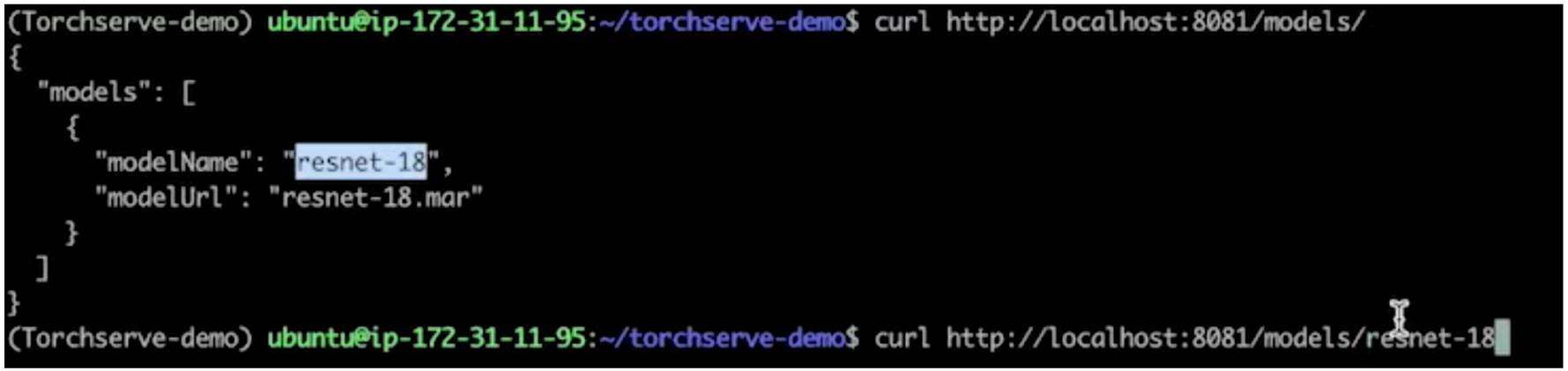


TorchServe

- `torch-model-archiver --model-name resnet-18 --version 1.0 --serialized-file resnet-18.pt --extra-files ./serve/examples/image_classifier/index_to_name.json --handler image_classifier`
- `mkdir model_store`
- `mv resnet-18.mar model_store/`
- `torchserve --start --model-store model_store --models resnet-18=resnet-18.mar --ncs`
- `curl http://127.0.0.1:8080/predictions/resnet-18 -T ./serve/examples/image_classifier/kitten.jpg`

TorchServe

- https://pytorch.org/serve/grpc_api.html



TorchServe

```
(Torchserve-demo) ubuntu@ip-172-31-11-95:~/torchserve-demo$ curl http://localhost:8081/models/
{
  "models": [
    {
      "modelName": "resnet-18",
      "modelUrl": "resnet-18.mar"
    }
  ]
}
(Torchserve-demo) ubuntu@ip-172-31-11-95:~/torchserve-demo$ curl http://localhost:8081/models/resnet-18
```


```
[
  {
    "modelName": "resnet-18",
    "modelVersion": "1.0",
    "modelUrl": "resnet-18.mar",
    "runtime": "python",
    "minWorkers": 16,
    "maxWorkers": 16,
    "batchSize": 1,
    "maxBatchDelay": 100,
    "loadedAtStartup": true,
    "workers": [
      {
        "id": "9000",
        "startTime": "2021-09-03T05:37:40.960Z",
        "status": "READY",
        "memoryUsage": 221454336,
        "pid": 23062,
        "gpu": false,
        "gpuUsage": "N/A"
      },
      {
        "id": "9001",
        "startTime": "2021-09-03T05:37:40.962Z",
        "status": "READY",
        "memoryUsage": 221167616,
        "pid": 23082,
        "gpu": false,
        "gpuUsage": "N/A"
      },
      {
        "id": "9002"
```

TorchServe

```
curl http://127.0.0.1:8080/predictions/resnet-18 -T ./serve/examples/image_classifier/kitten.jpg
```

```
(Torchserve-demo) ubuntu@ip-172-31-11-95:~/torchserve-demo$ curl http://127.0.0.1:8080/predictions/resnet-18 -T
{
  "tabby": 0.40864500403404236,
  "tiger_cat": 0.3506149351596832,
  "Egyptian_cat": 0.12609612941741943,
  "lynx": 0.023326952010393143,
  "bucket": 0.011855207383632666
}(Torchserve-demo) ubuntu@ip-172-31-11-95:~/torchserve-demo$
```